Un'introduzione alla programmazione di Algoritmi Genetici con Mathematica

Alessandro Vicini
Universita' degli Studi di Milano

Programma

- Per quale tipo di problemi sono utili gli Algoritmi Genetici (AG)
  Analogie (terminologiche) con la biologia
- Schema generale di un AG ed esempi di operatori genetici
- Perche' un AG puo' convergere rapidamente a una soluzione ottimale di un problema dato (schema's theorem)
- Il dilemma del prigioniero (torneo)
- Esempio di rete neurale
  Allenamento di reti neurali con il metodo della back-propagation
  Allenamento tramite un AG

Quando e' utile un Algoritmo Genetico?

- spazio delle soluzioni troppo grande per essere analizzato in modo esauriente
- la funzione che valuta le soluzioni ha piu' di un massimo locale
- esiste un'idea qualitativa di "soluzione ottimale", ma non una procedura nota a priori per determinarla

ricerca di dati in un database: no

ricerca di una soluzione ottimale gia' codificata (8-puzzle): +o-

ricerca di una soluzione non nota a priori, sia stato finale che procedura per raggiungerlo (protein folding, strategia per giocare al dilemma del prigioniero) si'

- Intuitivamente un AG implementa il processo di selezione naturale

data una popolazione, dati i criteri di selezione, si isolano, di generazione in generazione, gli individui "migliori", che si candidano come soluzione ottimale.

- Non e' detto che un AG trovi la soluzione migliore del problema

adattivo (se la soluzione dipende da molti parametri, la strategia di ricerca della soluzione puo' variare in regioni diverse dello spazio dei parametri; capacita' di rispondere a una variazione dell'ambiente)

ottimizzazione (soluzioni "accettabili" trovate rapidamente nel problema del commesso viaggiatore)

Analogie (terminologiche) con la biologia

{Individuo: realizzazione esplicita di una possibile soluzione del problema
{Cromosoma: (genotipo) codifica, di solito come una stringa di caratteri, tutte le informazioni necessarie a identificare un individuo

Geni: i singoli caratteri presenti nel cromosoma; la loro posizione puo' in certi casi determinare la loro funzione

Alleli: i valori che un singolo carattere (gene) puo' assumere

Popolazione: insieme di individui (inizialmente generato in modo casuale)

Fenotipo: in certi AG un individuo, specificato da un dato genotipo, e' selezionato in base a caratteristiche "esterne"

Riproduzione: ad ogni generazione una nuova popolazione viene generata
la scelta degli individui genitori si basa su criteri di selezione
la riproduzione puo' essere aploide o diploide

Selezione: criteri di scelta dei genitori (fitness-proportionate, elitismo,...)

Cross-over: operazione che consiste nel dividere i cromosomi di due individui in uno o piu' punti e nello scambiarli, dando luogo a due nuovi individui

Mutazione: operazione che consiste nel mutare in modo casuale il valore di uno o piu' geni

Schema generale di un Algoritmo Genetico

1) Codifica di un individuo, i.e. strutturazione e scrittura del cromosoma
2) Definizione della funzione di fitness con cui valutare ciascun individuo
3) Definizione dei criteri di selezione dei genitori (utilizzano la funzione di fitness)
4) Generazione (casuale) di una popolazione iniziale, di N individui
5) Valutazione della fitness di ciascun individuo
6) Selezione dei genitori: 6a) generazione dei figli attraverso il meccanismo di cross-over,
   6b) eventuale applicazione di operatori di mutazione
6) Si ripete quindi dal punto 5)
7) Dopo un certo numero di generazioni si completa un run;
   gli individui con la fitness piu' alta sono le soluzioni del problema

E' importante verificare l'indipendenza delle sequenze di numeri casuali utilizzati in diversi run

Codifica

Dato un problema e' necessario decidere come rappresentare la soluzione.

- E' sempre possibile ridurre qualunque espressione, formula, procedura, etc.
a una sequenza di bits ("geni") aventi come valore (0 e 1) "alleli"
Chiamiamo questa sequenza "cromosoma" e la identifichiamo con un individuo
es.: {0,0,1,0,1,1,0,1,...,0,1}
- In generale la rappresentazione binaria puo' non essere la piu' efficace per descrivere un individuo: il DNA dell'uomo e' codificato da un numero minore di "geni", ciascuno con un maggior numero di alleli (i 20 amminoacidi) es.: {A,D,E,C,B,D,A,A,E,B,B,...,E}
(cfr. commento dopo lo Schema's theorem)
- Oppure e' possibile che ogni gene assuma un valore reale.
- Nel caso di procedure, una rappresentazione ad albero puo' essere vantaggiosa
(e Mathematica fornisce l'ambiente adatto).

Non sempre un cromosoma deve essere di dimensione fissata.

Fitness

Lo spazio delle soluzioni e' costituito dall'insieme di tutti gli individui che possono essere generati, secondo le nostre convenzioni di rappresentazione del cromosoma.
Il processo di selezione si basa su una funzione di fitness, che classifica gli individui della popolazione: e' una funzione a valori reali, definita sullo spazio delle soluzioni.
Es.: il numero di bits pari a 1 nel cromosoma.

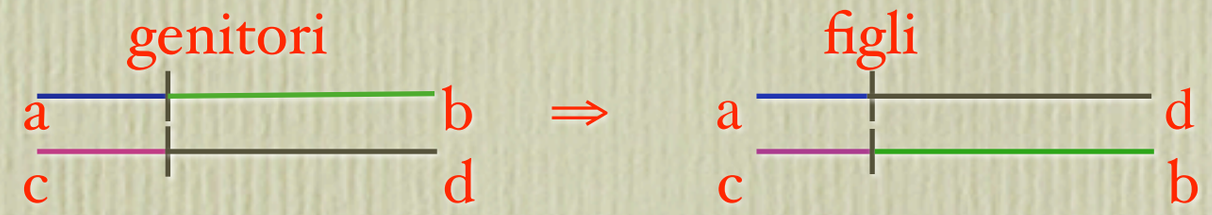
{0,1,0,1,0,1} → f=3
{1,1,0,1,1,1} → f=5
{0,0,0,1,0,0} → f=1

Dalla forma di questa funzione dipende la velocita' di convergenza verso la soluzione ottimale.
Nel caso in cui la funzione abbia al piu' un massimo nello spazio delle soluzioni, esistono metodi piu' efficaci rispetto agli AG per trovarlo.
Nel caso invece di forme funzionali piu' complicate, un AG puo' avere delle chances di trovare soluzioni soddisfacenti, in modo efficiente.

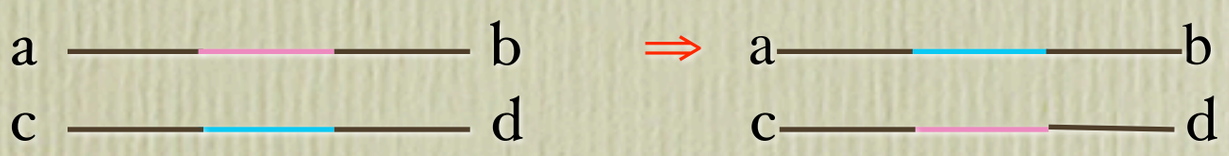
Crossover e mutazione

Cross-over (aploide)(punto singolo): dati due cromosomi genitori, si sceglie in modo casuale un punto lungo il cromosoma, e si scambia la prima parte del primo con la prima parte del secondo cromosoma; lo scambio puo' avvenire condizionato da un'estrazione con probabilita' tipica p_c=0.7

cromosoma ——————— = {0,1,1,0,1,0,...,0,1} la scelta di un punto divide la stringa in due



Cross-over (aploide)(due punti): dati due cromosomi genitori, si scelgono in modo casuale due punti lungo il cromosoma, e si scambia la parte compresa tra i due punti.



Mutazione : si percorre tutto il cromosoma, bit per bit, e si cambia con probabilita' tipicamente bassa (e.g. p_m=0.001) il valore corrente di ciascun gene

Selezione

Problematica generale:

- un AG e' utile quando e' necessario esplorare un ampio spazio delle soluzioni
- nel caso di funzioni con molti massimi locali, e' opportuno avere meccanismi che permettano di evitare di arenarsi su uno di questi
- la pressione nel processo di selezione esprime la difficolta' che gli individui migliori incontrano nel riprodursi
- la varianza della popolazione esprime il grado di diversita' e quindi la possibilita', tramite cross-over, di esplorare nuove soluzioni

Fitness-proportionate : si estraggono, con probabilita' proporzionale alla loro fitness gli individui che devono riprodursi

Pro : si consente a tutti gli individui di avere una possibilita' riproduttiva

Contro : e' possibile che nelle fasi iniziali (nelle prime generazioni), quando la varianza della popolazione e' alta, gli individui migliori predominino riproducendosi molto piu' rapidamente degli altri, portando l'AG a una "convergenza prematura", impedendo altre esplorazioni nello spazio delle soluzioni

Sigma-scaling : pressione selettiva costante nel run: la probabilita' di riproduzione e' legata oltre che alla fitness dell'individuo, anche alla media e alla varianza della fitness della popolazione

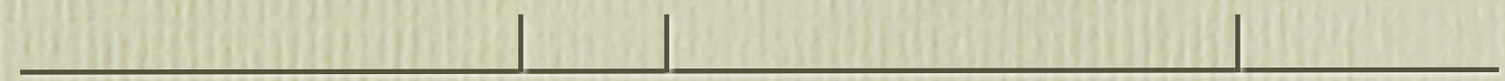
Elitismo : si sceglie di preservare, a ogni generazione, alcuni degli individui migliori, che potrebbero andare persi a causa di cross-over e/o mutazione

Ranking : si classificano gli individui, perdendo traccia della differenza assoluta in fitness; abbassa la pressione quando la varianza e' grande, e tiene la pressione alta quando la varianza e' piccola; funziona bene in caso di funzioni di fitness con "rumore"

Esempi di selezione

popolazione = {I1,I2,I3,I4}
fitness = {12, 4, 16, 8} fitness media = 10 fitness totale popolazione= 40
frazione della popolazione che ciascun individuo si aspetta di generare
={0.3, 0.1, 0.4, 0.2}

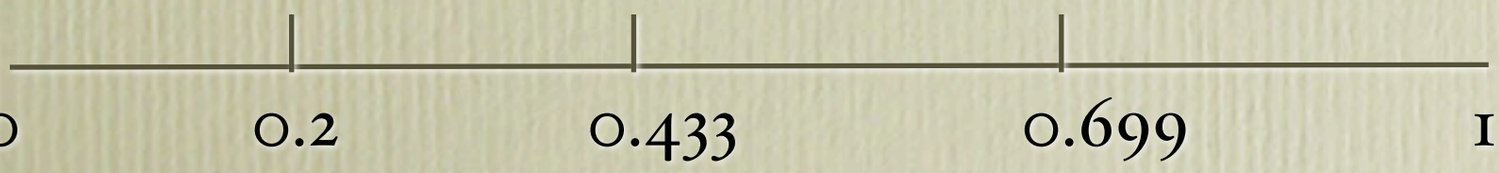
❖ suddivido l'intervallo [0,1] secondo queste frazioni (fitness-proportionate)



se getto 1000 punti in [0, 1] mi aspetto che 300 cadano nel primo intervallo
100 nel secondo, 400 nel terzo e 200 nel quarto
per stabilire in quale intervallo e' caduto un numero casuale, e scegliere quindi il genitore, uso la funzione di Mathematica Select

☻ ordino gli individui in base alla fitness e assegno loro una fetta dell'intervallo [0,1] che dipende solo dalla posizione, ma non dalla fitness assoluta ranking: {I2,I4,I1,I3}
assegno fette di [0,1] secondo la legge $f_i = \frac{2}{N(max+min)}\left[min + (max-min)\frac{rank(i)-1}{N-1}\right]$

es: min = 0.8 max=2-min N=4 f = {0.2, 0.233, 0.266, 0.3}



Esercizi

1) Implementare un AG, con selezione fitness-proportionate, 100 individui, p_c=0.7 e p_m=0.001. La funzione di fitness sia f(x)= numero di 1 che appaiono nella stringa x, lunga 20 bits. Compiere 20 runs e misurare la generazione media in cui la stringa con tutti 1 e' scoperta. Studiare la dipendenza da cross-over e mutazione.

2) Implementare un AG con selezione fitness-proportionate, 100 individui, cross-over a punto singolo p_c=0.7 e mutazione p_m=0.001. La funzione di fitness f(x) sia il numero intero rappresentato in notazione binaria dalla stringa x, di lunghezza 20. Per 100 generazioni disegnare la fitness dell'individuo migliore.
Come dipende da p_c e p_m?

Schema's theorem

Gli AG selezionano le strutture "buone" all'interno del cromosoma

- Un sottoinsieme di tutte le possibili strutture sono gli Schemi
  Nel caso binario, si intende una sequenza di bits formata da 0,1 e *
  Es.: 1****1, oppure *1, oppure **, oppure 0****
- Si dice che 10001 e' un'istanza dello schema 1***1
  Una stringa di *l* bits e' un'istanza di 2^*l* schemi diversi
  (In una stringa con *l* bits ci sono 3^*l* schemi ma 2^(2^(*l*-1)) sottoinsiemi distinti)
- Valutare la fitness di una stringa implicitamente valuta la fitness media di 2^*l* schemi, dove la fitness media e' definita come la media delle fitness delle istanze dello schema
  Questo parallelismo implicito dovrebbe favorire nella popolazione le istanze di schemi che hanno fitness sopra media.

Sia H uno schema, $m(H,t)$ il numero di istanze di H al tempo t, $\hat{u}(H,t)$ la fitness media di H al tempo t, $f(x)$ la fitness di una stringa x, $\bar{f}(t)$ la fitness media della popolazione al tempo t. Vogliamo calcolare $E(m(H,t+1))$, il numero di istanze di H al tempo t+1

$E(m(H,t+1)) = \sum_{x \in H} f(x)/\bar{f}(t) = m(H,t)\hat{u}(H,t)/\bar{f}(t)$ numero di figli istanze di H

$S_c(H) \geq 1 - p_c(\frac{d(H)}{l-1})$ probabilita' di sopravvivenza di H per cross-over

d(H) lunghezza intrinseca compresa tra 0 e l-1

Schema's theorem (2)

$(1-p_m)^{o(H)}$ Probabilita' di sopravvivenza di H per una mutazione

$$E(m(H,t+1)) \geq \frac{\hat{u}(H,t)}{\bar{f}(t)} m(H,t) \left(1 - p_c(\frac{d(H)}{l-1}\right) [(1-p_m)^{o(H)}]$$

L' AG non calcola $\hat{u}(H,t)$ ma ne dipende, al punto che solo gli schemi con una fitness superiore agli effetti distruttivi di cross-over e mutazione, sopravvivono e prolificano

E' chiaro che gli schemi con lunghezza intrinseca *d(H)* grande danno luogo a un rapporto con il numero totale *l-1* di bits della stringa prossimo a 1, e subiscono maggiormente gli effetti del cross-over
Al contrario, schemi corti dovrebbero in generale sopravvivere meglio a cross-over e mutazione.

Si dice che la codifica binaria, allungando i cromosomi, offra una maggiore possibilita' di sfruttare gli effetti di parallelismo implicito degli AG
E' vero pero' che se gli schemi brevi sono favoriti, allora una parametrizzazione "lunga" offre il fianco agli effetti distruttivi del cross-over

Parametri, pressione, convergenza

Fitness Con voti appiattiti su un unico valore, e' difficile distinguere bravi e asini
Con uno spettro di valori troppo ampio e una selezione fitness-proportionate, c'e' il rischio che i piu' bravi prendano troppo vantaggio, prolifichino e invadano la popolazione, bloccando l'AG su un massimo che potrebbe non essere assoluto

Mutazione Nel caso la funzione di fitness abbia molti massimi locali, e' utile la mutazione che consente di esplorare un maggior numero di regioni dello spazio delle soluzioni

Crossover Il ruolo del crossover e' piu' legato alla natura del problema e alla sua codifica. Nel caso si riconoscano nella soluzione sottoinsiemi con un effetto preciso per innalzare la fitness, questi vanno preservati e al tempo stesso va migliorato il resto della stringa (cfr. cambio delle carte a poker)

Fitness statica e variabile Nel caso i test con cui si misura la fitness non siano troppo severi l'AG puo' fermarsi su un massimo locale, non necessariamente ottimale. Una crescita nella difficolta' dei test puo' servire a mantenere la necessaria pressione e spingere l'AG a trovare soluzioni ottimali.

Il dilemma del prigioniero

Alice e Bob sono stati arrestati e rinchiusi in due celle senza possibilita' di comunicazione.
Ad Alice viene proposto quanto segue:

☺ se confessa accusando Bob viene messa in liberta' provvisoria e Bob viene richiuso per 5 anni

☻ se anche Bob confessa entrambe le confessioni risultano capziose e ciascuno viene rinchiuso per 4 anni

☹ se nessuno dei due confessa, vengono rinchiusi entrambi per due anni

Alice sa che a Bob e' stata fatta la stessa offerta.

Quale tipo di comportamento conviene a ciascuno dei due?
Collaborare con il compagno (ovvero non confessare) o tradirlo?
I punti in tabella sono dati da 5-#anni di prigione

	Collabora	Tradisce
Collabora	(3,3)	(0,5)
Tradisce	(5,0)	(1,1)

Ci si puo' porre il problema di trovare una strategia che permetta di giocare una serie di partite basate su questa tabella, avendo memoria dei risultati delle ultime tre partite, in modo da massimizzare il risultato.
La memoria delle ultime tre partite rappresenta il comportamento dell'avversario

Il dilemma del prigioniero

Una strategia di gioco puo' essere formalizzata nel seguente modo (rispetto alla partita precedente): per ogni coppia di scelte (in totale sono 4) fatte da Alice e Bob nella partita precedente stabilire la mossa corrispondente; per esempio:
(C,C) → C, (C,T) → T, (T,C) → C, (T,T) → T
considerando ((C,C),(C,T),(T,C),(T,T)) l'ordine canonico delle 4 possibili giocate
la strategia puo' essere abbreviata con CTCT; ci sono 2^4=16 strategie possibili
nel caso di una memoria delle ultime tre partite, ci sono 4^3=64 giocate e quindi 2^64 strategie possibili
Lo spazio delle soluzioni e' enorme. Come si puo' scrivere un algoritmo che impari a conoscere il proprio avversario e che elabori una strategia vincente?

Un AG puo' dunque essere impostato nel seguente modo:
1) Una strategia, un individuo (cromosoma) e' una stringa di 64 bits, 0 oppure 1
2) Si genera una popolazione; ciascun individuo viene fatto giocare per M partite contro un avversario; la sua fitness e' data dal punteggio medio ottenuto
3) Si selezionano i genitori sulla base della fitness ottenuta
4) Con cross-over e mutazione si genera la nuova popolazione

TIT FOR TAT e altre strategie

TIT FOR TAT e' una strategia che consiste nell'offrire collaborazione nella prima giocata del DdP; successivamente replica l'ultima mossa dell'avversario.

Salita piu' ripida Metodo ("procedurale") di ricerca di massimi della funzione di fitness. Data una stringa iniziale la si chiama *best* e se ne valuta la fitness; quindi si flippa sistematicamente ogni bit e si valuta la fitness della nuova stringa; se c'e' un aumento, la stringa viene definita *best* e si riparte dal primo bit; altrimenti, arrivati in fondo, si salva la stringa e si ricomincia da capo. Dopo avere generato in questo modo una popolazione, si sceglie l'individuo migliore.

Salita con mutazione random Si generi a caso una stringa e la si definisca *best*. Si scelga a caso un bit e lo si flippi; se la nuova stringa ha fitness migliore della precedente, diventa *best* . Si continua finche' una stringa ottimale o un numero massimo di mutazioni e' stato effettuato.